

PROGRAMACIÓN APLICADA

Guía del estudiante · Clase 01

PARKFLOW 360

Del problema del cliente al repositorio y backlog inicial

PROPÓSITO

Aprender a transformar un problema ambiguo en una primera versión entendible del proyecto y dejar esa comprensión registrada en Git. Al finalizar deberás poder explicar el problema, actores, alcance, exclusiones, requisitos, reglas e historias, y demostrar un repositorio con README, visión, glosario, backlog y primer commit.

Dato	Definición
Duración de referencia	90 minutos
Modalidad	Explicación + práctica guiada + trabajo en equipo
Herramientas	IntelliJ IDEA o editor, PowerShell/terminal, Git, GitHub, navegador
Todavía NO usamos	Spring Boot, PostgreSQL, React, React Native
Evidencia	README + visión + glosario + backlog + commit + repositorio remoto si es posible

IDEA CLAVE

Hoy no ganas puntos por escribir código más rápido. Ganas comprensión. Si no puedes explicar qué estás construyendo, todavía no estás listo para diseñar tablas ni controllers.

1. Qué debes ser capaz de hacer al terminar

- Diferenciar problema, solución, actor, alcance, exclusión, requisito funcional, regla de negocio, historia de usuario y criterio de aceptación.
- Leer la ficha de un proyecto sin inventar requisitos ni reducir el alcance por comodidad.
- Detectar conceptos de dominio importantes y empezar un lenguaje común con tu equipo.
- Crear un repositorio local Git desde una carpeta vacía.
- Organizar documentación inicial sin mezclar todo en un único archivo.
- Escribir un README v0.1 comprensible por una persona que no estuvo en clase.
- Crear un backlog inicial priorizado y diferenciar historias de usuario de tareas técnicas.
- Hacer un commit descriptivo y explicar qué evidencia deja.
- Transferir exactamente el mismo proceso mental a tu proyecto asignado sin copiar el dominio ParkFlow.

AUTOCONTROL

Si puedes hacer los pasos mecánicos pero no puedes explicar por qué existen README, visión, glosario y backlog, todavía no has terminado la Clase 01.

2. Conceptos que debes dominar antes de tocar código

Concepto	Qué significa	Ejemplo correcto	Error típico
Problema	Dolor, riesgo o ineficiencia actual.	"La disponibilidad no es confiable y se pierden reservas."	"Hacer una app."
Actor	Rol o parte externa con una interacción o responsabilidad relevante.	Operador, Cliente, Supervisor.	"Tabla Usuario".
Alcance	Lo que el MVP sí promete resolver.	Reservas, ocupación, entradas/salidas.	"Todo lo que pueda necesitar un estacionamiento".
Exclusión	Lo que declaramos explícitamente fuera.	IoT, cámaras/LPR, pasarela bancaria real.	"Ya veremos después".
Requisito funcional	Capacidad observable que el sistema debe ofrecer.	Crear una reserva.	"Usar Java 21".
Regla de negocio	Condición del dominio que gobierna una operación.	Un espacio bloqueado no puede reservarse.	"El botón debe ser azul".
Historia de usuario	Necesidad desde un actor y su valor.	Como cliente, quiero consultar disponibilidad para reservar.	"Crear tabla reservas".
Criterio de aceptación	Condición verificable que demuestra que una historia funciona.	Dado un espacio bloqueado, no debe aparecer disponible.	"Funciona bien".

REGLA DE ORO

Requisito = QUÉ capacidad existe. Regla de negocio = QUÉ condición debe respetar. Criterio de aceptación = CÓMO comprobamos que funciona.

2.1 Mini ejercicio mental

- “Un espacio en mantenimiento no debe poder reservarse”. ¿Es requisito, regla o criterio?
- “Como supervisor, quiero ver la ocupación actual para detectar saturación”. ¿Qué tipo de artefacto es?
- “El sistema usará PostgreSQL”. ¿Es problema del cliente o decisión/requisito técnico?
- “No integraremos cámaras en el MVP”. ¿Qué representa?

RESPUESTAS

1) Regla de negocio. 2) Historia de usuario. 3) Requisito/decisión técnica, no problema del cliente. 4) Exclusión.

3. Comprender ParkFlow 360 antes de construirlo

Situación de trabajo: una empresa opera uno o más estacionamientos urbanos y actualmente usa hojas de cálculo, tickets manuales y mensajería. Esto produce inconsistencias de disponibilidad, reservas superpuestas, dificultad para localizar vehículos, pérdida de trazabilidad y poca información para supervisión.

Categoría	Versión inicial
Problema	Disponibilidad poco confiable, reservas incompatibles y trazabilidad incompleta.
Actores	Administrador, Operador, Cliente/Conductor, Mantenimiento, Supervisor.
Núcleo del MVP	Estacionamientos, zonas, espacios, clientes, vehículos, disponibilidad, reservas, entrada/salida, estancias, bloqueos, incidencias, indicadores.
Fuera del MVP	IoT, cámaras/LPR, barreras físicas, GPS, pagos bancarios reales, facturación fiscal, IA para decidir reservas o precios.

3.1 Preguntas que debes poder responder

- ¿Qué dolor existe hoy?
- ¿Quién registra físicamente una entrada?
- ¿Quién crea o consulta una reserva?
- ¿Qué significa que un espacio esté “disponible”?
- ¿Un espacio físicamente libre siempre debe ser reservable?
- ¿Qué información histórica no debería borrarse?
- ¿Qué funcionalidades atractivas debemos excluir para proteger el MVP?

4. Laboratorio: crea el repositorio desde cero

IMPORTANTE

Hazlo desde una carpeta vacía. No descargues un ZIP “resuelto”. El objetivo es comprender cada evidencia que aparece.

Paso 1 — comprobar Git

Copia y pega:

```
git --version
```

Debes ver algo parecido a: git version 2.x.x. Si Windows indica que git no se reconoce, Git no está disponible en PATH. Corrige esto antes de continuar.

Paso 2 — crear la carpeta raíz

PowerShell:

```
mkdir parkflow-360
cd parkflow-360
Get-Location
```

La carpeta raíz representa el producto completo. Más adelante contendrá backend, web, mobile y docs. Hoy sólo construiremos la base documental y de versionado.

Paso 3 — inicializar Git

Copia y pega:

```
git init
git branch -M main
git status
```

Comando	Qué hace	Qué debes poder explicar
git init	Convierte la carpeta en repositorio local.	Qué cambia al aparecer .git.
git branch -M main	Define/renombra la rama principal como main.	Qué es una rama y por qué existe main.
git status	Muestra estado del área de trabajo y staging.	Qué archivos están modificados, preparados o sin seguimiento.

Paso 4 — crear estructura documental

PowerShell — copia y pega:

```
mkdir docs
mkdir docs\01-vision
mkdir docs\02-requirements
mkdir docs\03-decisions
New-Item README.md -ItemType File
New-Item docs\01-vision\vision-v0.1.md -ItemType File
New-Item docs\01-vision\glossary-v0.1.md -ItemType File
New-Item docs\02-requirements\backlog-v0.1.md -ItemType File
New-Item docs\03-decisions\README.md -ItemType File
tree /F
```

Estructura esperada:

```

parkflow-360
├── README.md
├── docs
│   ├── 01-vision
│   │   ├── vision-v0.1.md
│   │   └── glossary-v0.1.md
│   ├── 02-requirements
│   │   └── backlog-v0.1.md
│   └── 03-decisions
│       └── README.md

```

5. Contenido exacto de los archivos

MÉTODO

No pegues todo sin leer. Copia un bloque, léelo, identifica qué información representa, guárdalo y recién pasa al siguiente archivo.

5.1 README.md

```
# ParkFlow 360
```

Sistema académico full-stack para la gestión de estacionamientos, reservas, accesos, ocupación e incidencias.

1. Problema

Actualmente la operación depende de hojas de cálculo, tickets manuales y mensajería. Esto puede producir inconsistencias de disponibilidad, reservas superpuestas, pérdida de trazabilidad de entradas y salidas y poca visibilidad para supervisión.

2. Objetivo del MVP

Construir un sistema web y móvil que permita administrar estacionamientos, zonas y espacios; registrar clientes y vehículos; consultar disponibilidad; gestionar reservas; registrar entradas y salidas; controlar ocupación, bloqueos e incidencias; y mostrar indicadores operativos básicos.

3. Actores principales

- Administrador
- Operador
- Cliente / conductor
- Personal de mantenimiento
- Supervisor

4. Alcance inicial

- Estacionamientos, zonas y espacios
- Clientes y vehículos
- Disponibilidad
- Reservas
- Entradas, salidas y estancias
- Bloqueos e incidencias
- Tarifas y registro operativo de pago
- Dashboard e historial básico

5. Fuera de alcance

- Sensores IoT

- Cámaras y lectura automática de placas
- Barreras físicas
- Pasarela bancaria real
- Facturación fiscal
- GPS en tiempo real
- IA para autorizar reservas o calcular precios

6. Stack objetivo del semestre

- Backend: Java 21 + Spring Boot
- Base de datos: PostgreSQL + Flyway
- Web: React + TypeScript
- Móvil: React Native + TypeScript
- Pruebas API: Postman
- Contenedores: Docker / Docker Compose
- Versionado: Git + GitHub
- CI: GitHub Actions
- IA: Spring AI, únicamente como capacidad complementaria

7. Estado actual

Clase 01: comprensión del problema, alcance, lenguaje inicial del dominio y backlog v0.1. Todavía no existe código de aplicación.

8. Documentación

- `docs/01-vision/vision-v0.1.md`
- `docs/01-vision/glossary-v0.1.md`
- `docs/02-requirements/backlog-v0.1.md`
- `docs/03-decisions/`

9. Regla de trabajo

Cada cambio importante debe ser comprensible, trazable y defendible. El repositorio es la fuente de verdad del proyecto.

PREGUNTA DE CONTROL

Si otra persona abre únicamente el README, ¿puede entender qué problema resolvemos, qué sí haremos, qué no haremos y en qué estado está el proyecto?

5.2 vision-v0.1.md

Visión v0.1 – ParkFlow 360

1. Contexto

Una empresa opera uno o más estacionamientos urbanos y necesita reemplazar controles manuales dispersos por una plataforma única.

2. Problema

La operación actual puede generar espacios marcados como libres cuando están ocupados, reservas incompatibles, dificultad para localizar vehículos, pérdida de trazabilidad de entradas y salidas y escasa información para supervisión.

3. Objetivo

Centralizar la operación del estacionamiento y mantener una representación confiable de disponibilidad, reservas, ocupación, entradas, salidas, bloqueos e incidencias.

4. Actores

Administrador

Configura datos maestros, usuarios, estacionamientos, zonas, espacios, tarifas y parámetros.

```

### Operador
Registra entrada, salida, validación de reserva y operación diaria.

### Cliente / conductor
Registra o consulta vehículos, busca disponibilidad, crea/cancela reservas permitidas y consulta su estancia.

### Personal de mantenimiento
Registra o gestiona bloqueos que vuelven temporalmente indisponible un espacio.

### Supervisor
Consulta ocupación, incidencias, excepciones e indicadores.

## 5. Alcance del MVP
1. Gestión de estacionamientos, zonas y espacios.
2. Gestión de clientes y vehículos.
3. Consulta de disponibilidad.
4. Creación y gestión de reservas.
5. Registro de entrada y salida.
6. Control de estancia activa y estado del espacio.
7. Bloqueos de espacio e incidencias.
8. Tarifas y registro operativo simple de pago.
9. Historial y dashboard operativo.

## 6. Exclusiones
No incluye IoT, cámaras, lectura automática de placas, barreras físicas, pasarela bancaria real, facturación fiscal, GPS en tiempo real ni decisiones automáticas de negocio mediante IA.

## 7. Éxito inicial del proyecto
El proyecto será exitoso cuando pueda demostrar, con datos persistidos, un flujo coherente desde la búsqueda de disponibilidad hasta la reserva, entrada, estancia y salida, consumido por web y móvil sobre el mismo backend.

```

5.3 glossary-v0.1.md

Glosario v0.1 – ParkFlow 360

Término	Significado inicial
Estacionamiento	Instalación física administrada por el sistema.
Zona	Agrupación lógica o física de espacios dentro de un estacionamiento.
Espacio	Lugar individual donde puede ubicarse un vehículo.
Disponibilidad	Condición por la cual un espacio puede ser utilizado o reservado en un intervalo determinado.
Cliente	Persona que utiliza el servicio de estacionamiento.
Vehículo	Vehículo registrado y asociado a un cliente según las reglas del MVP.
Reserva	Compromiso temporal de disponibilidad para un vehículo/cliente en un intervalo definido.
Entrada	Evento que registra el ingreso operativo de un vehículo.
Estancia	Periodo abierto entre la entrada y la salida de un vehículo.
Salida	Evento que finaliza una estancia.
Bloqueo	Indisponibilidad temporal de un espacio por mantenimiento u otra causa autorizada.
Incidencia	Hecho excepcional ocurrido durante la operación que debe quedar registrado.
Tarifa	Regla o valor operativo usado para calcular un importe simple.
Ocupación	Estado que indica que un espacio está siendo utilizado por una estancia activa.

> Este glosario es v0.1. Los significados pueden refinarse cuando aparezcan nuevas reglas, pero los cambios deberán quedar documentados.

5.4 backlog-v0.1.md

```
# Backlog v0.1 – ParkFlow 360

## Convención de prioridad
- P0: imprescindible para el flujo crítico del MVP.
- P1: importante para completar la operación.
- P2: complemento posterior.

### PF-001 – Registrar estacionamiento
**Prioridad:** P0
Como administrador, quiero registrar un estacionamiento para poder organizar sus zonas y espacios.

### PF-002 – Registrar zonas y espacios
**Prioridad:** P0
Como administrador, quiero crear zonas y espacios para representar la capacidad física del estacionamiento.

### PF-003 – Registrar cliente y vehículo
**Prioridad:** P0
Como operador, quiero registrar un cliente y su vehículo para poder asociarlos a reservas y estancias.

### PF-004 – Buscar disponibilidad
**Prioridad:** P0
Como cliente, quiero consultar espacios disponibles para un intervalo para poder elegir una opción válida.

### PF-005 – Crear reserva
**Prioridad:** P0
Como cliente, quiero crear una reserva para asegurar disponibilidad durante un intervalo.

### PF-006 – Registrar entrada
**Prioridad:** P0
Como operador, quiero registrar la entrada de un vehículo para abrir una estancia y reflejar ocupación real.

### PF-007 – Registrar salida
**Prioridad:** P0
Como operador, quiero registrar la salida de un vehículo para cerrar su estancia y liberar el espacio.

### PF-008 – Bloquear espacio
**Prioridad:** P1
Como personal autorizado, quiero bloquear un espacio para mantenimiento para impedir que aparezca disponible mientras dure el bloqueo.

### PF-009 – Registrar incidencia
**Prioridad:** P1
Como operador, quiero registrar una incidencia para conservar trazabilidad de situaciones excepcionales.

### PF-010 – Consultar dashboard
**Prioridad:** P1
Como supervisor, quiero consultar ocupación, disponibilidad, reservas e incidencias para entender el estado operativo.
```

ATENCIÓN

Una historia no es “crear tabla”, “hacer endpoint” ni “hacer formulario”. Es una necesidad que genera valor para un actor. Las tareas técnicas aparecerán después para implementar la historia.

5.5 docs/03-decisions/README.md

Decisiones técnicas y de producto

Esta carpeta almacenará decisiones relevantes del proyecto (ADR o notas equivalentes).
Cada decisión importante deberá indicar:

1. contexto;
2. problema o decisión;
3. alternativas consideradas;
4. decisión tomada;
5. consecuencias conocidas.

En la Clase 01 todavía no se registra una decisión arquitectónica formal porque aún no se ha iniciado el diseño técnico detallado.

6. Registrar la evidencia en Git

Ejecuta en este orden:

```
git status
git add .
git status
git commit -m "docs: initialize ParkFlow vision and backlog"
git log --oneline -5
```

Paso	Qué debes observar
Primer git status	Archivos nuevos todavía sin seguimiento.
git add .	Los archivos pasan al staging area.
Segundo git status	Los archivos aparecen preparados para commit.
git commit	Se crea una unidad de cambio identificable.
git log --oneline -5	Puedes ver el historial y el mensaje descriptivo.

NO MEMORICES

Un commit no es “guardar”. Es registrar una unidad comprensible de evolución del producto. Debes poder explicar qué cambió y por qué.

7. Publicar en GitHub

1. En GitHub selecciona New repository.
2. Nombre: parkflow-360.
3. Descripción: Proyecto docente modelo de Programación Aplicada 2026-2.
4. Usa la visibilidad definida por el docente.

5. NO marques “Add a README” porque ya existe localmente.
6. Crea el repositorio y copia la URL HTTPS.

Reemplaza USUARIO por tu cuenta:

```
git remote add origin https://github.com/USUARIO/parkflow-360.git
git push -u origin main
git remote -v
git status
```

SEGURIDAD

Nunca pegues contraseñas, tokens o archivos .env reales en README ni los subas a Git. Si aparece un problema de autenticación, usa el mecanismo configurado por GitHub/Git Credential Manager.

8. Ahora hazlo en tu proyecto asignado: espejo, no copia

El objetivo no es transformar todos los proyectos en ParkFlow. Debes copiar el proceso de razonamiento y usar exclusivamente los actores, reglas y conceptos definidos en la ficha oficial de tu proyecto.

Debes producir	Mínimo
README v0.1	Problema, objetivo, actores, alcance, exclusiones, stack, estado actual.
Glosario v0.1	Al menos 10 términos reales de tu dominio.
Backlog v0.1	Al menos 8 historias priorizadas P0/P1/P2.
Git	Repositorio inicializado, historial visible y commit descriptivo.
Comprensión	Todos los integrantes pueden explicar el contenido, no sólo quien lo escribió.

PROHIBIDO

No inventes requisitos que contradigan la ficha oficial. No reduzcas el alcance porque “parece difícil”. No copies nombres ParkFlow si tu dominio usa citas, préstamos, órdenes, envíos, membresías, incidencias u otros conceptos.

8.1 Checklist de revisión del equipo

- ☐ El problema describe un dolor o necesidad, no “hacer un sistema”.
- ☐ Los actores coinciden con la ficha oficial.
- ☐ El alcance mantiene lo solicitado por el cliente académico.
- ☐ Las exclusiones evitan crecimiento descontrolado.
- ☐ El glosario usa lenguaje del dominio, no palabras como controller/repository.
- ☐ Las historias tienen actor, necesidad y valor.
- ☐ No todo está marcado P0.
- ☐ Todo está dentro del repositorio oficial.
- ☐ Cada integrante sabe explicar por qué existe cada archivo.

9. Errores típicos que te pueden hacer perder comprensión

Error	Por qué está mal	Cómo corregirlo
Actor = Usuario	Confunde responsabilidad con persistencia.	Primero identifica interacción; luego veremos si necesita identidad.
Todo es P0	Si todo es urgente, no existe prioridad.	P0 sólo sostiene el flujo crítico.
Historia = crear tabla	Una tabla es implementación, no necesidad del actor.	Reformula desde actor + necesidad + valor.
Alcance lleno de tecnologías	Java/PostgreSQL no describen el problema del cliente.	Separa negocio de decisiones técnicas.
Glosario técnico	Controller/Repository no son lenguaje del estacionamiento.	Usa Reserva, Espacio, Estancia, Bloqueo, etc.
Commit “avance”	No explica qué cambio ocurrió.	Usa mensaje descriptivo.
Saltar a Spring Boot	Te hace diseñar sin comprender conceptos y reglas.	Termina primero la evidencia de Clase 01.

10. Prepárate desde hoy para la defensa individual

La materia exige que cada integrante pueda explicar el avance completo del equipo. Por eso, aunque otro compañero escriba un archivo, tú debes entenderlo y poder defenderlo.

Pregunta	Respuesta mínima defendible
¿Qué es alcance?	El compromiso de lo que el MVP sí resolverá.
¿Para qué sirve una exclusión?	Para fijar límites y evitar ambigüedad o crecimiento no controlado.
¿RF y RN son lo mismo?	No. RF = capacidad. RN = condición que gobierna esa capacidad.
¿Actor significa cuenta de usuario?	No necesariamente; actor representa responsabilidad/interacción.
¿Para qué sirve el glosario?	Para fijar significados compartidos del dominio.
¿Qué es una historia P0?	Una historia imprescindible para el núcleo del MVP.
¿Qué evidencia queda en Git?	Archivos versionados y un commit identificable en el historial.
¿Por qué aún no creamos tablas?	Porque primero debemos identificar conceptos y reglas antes de diseñar persistencia.

11. Autoevaluación de Clase 01

- ☐ Puedo explicar el problema de ParkFlow sin decir “hacer una app”.
- ☐ Puedo nombrar al menos cuatro actores y su responsabilidad.
- ☐ Puedo explicar alcance y exclusión con un ejemplo.
- ☐ Puedo diferenciar requisito, regla e historia.
- ☐ Puedo explicar por qué un glosario de dominio es útil.

- ☐ Puedo ejecutar git status y explicar la salida.
- ☐ Puedo explicar qué hace git add y qué hace git commit.
- ☐ Puedo identificar el primer commit en git log.
- ☐ Puedo explicar por qué README, visión, glosario y backlog no son el mismo artefacto.
- ☐ Puedo aplicar el patrón de ParkFlow a mi proyecto sin copiar sus términos.

SEMÁFORO PERSONAL

8–10 marcadas: listo para Clase 02. 5–7: repasa y practica. Menos de 5: vuelve a ejecutar la guía desde una carpeta vacía y explica cada paso en voz alta.

12. Trabajo obligatorio antes de la Clase 02

- Termina README v0.1, glosario y backlog inicial de tu proyecto usando la ficha oficial.
- Debe existir al menos un commit por integrante o evidencia equivalente de colaboración organizada.
- Relee la ficha y marca sustantivos, procesos, estados y restricciones.
- No diseñes todavía el DER definitivo.
- No crees tablas manualmente en PostgreSQL como procedimiento oficial.
- No saltes a JPA ni controllers.
- Llega preparado para identificar entidades, atributos, relaciones y cardinalidades.

13. Puente directo a la Clase 02

PREGUNTA DE APERTURA

En el glosario aparecen Espacio, Reserva, Vehículo y Estancia. ¿Cuáles necesitan identidad y persistencia? ¿Cómo se relacionan? ¿Qué datos serían atributos y qué reglas deberían convertirse en restricciones?

En la siguiente clase pasarás del lenguaje del dominio al modelado conceptual. Todavía no será “crear tablas porque sí”: aprenderás a justificar entidades, atributos, relaciones, cardinalidades y primeras reglas de integridad.

14. Hoja rápida de bolsillo

Si te preguntan...	Recuerda...
Problema	Dolor actual, no la solución.
Actor	Responsabilidad/interacción, no tabla.
Alcance	Qué sí hacemos.
Exclusión	Qué no hacemos.
RF	Capacidad.

RN	Condición.
Historia	Actor + necesidad + valor.
Criterio	Cómo verificamos.
README	Puerta de entrada al repositorio.
Glosario	Lenguaje común del dominio.
Backlog	Trabajo priorizado orientado a valor.
Git	Evidencia de evolución y colaboración.

15. Definition of Done de tu Clase 01

- ☐ El proyecto puede ser entendido leyendo sólo el README.
- ☐ El alcance no contradice la ficha oficial.
- ☐ Las exclusiones están escritas.
- ☐ Existe un lenguaje inicial de dominio.
- ☐ El backlog tiene historias priorizadas y no sólo tareas técnicas.
- ☐ Los archivos están versionados.
- ☐ Puedes explicar git status y git log.
- ☐ No hay secretos ni credenciales en Git.
- ☐ Puedes transferir el patrón a tu proyecto asignado.
- ☐ Puedes responder las preguntas de defensa sin depender de otro integrante.

16. Cómo estudiar esta clase de forma inteligente

- No memorices definiciones aisladas: da un ejemplo de tu propio proyecto para cada concepto.
- Repite el laboratorio una segunda vez sin mirar los comandos y luego verifica.
- Explícale a un compañero la diferencia entre requisito, regla y criterio.
- Abre un repositorio de otro integrante y trata de entenderlo sólo con el README.
- Practica una mini defensa de 3 minutos: problema → actores → alcance → exclusiones → backlog → evidencia Git.
- Si algo no puedes explicar, no lo marques como terminado.

CIERRE

La Clase 01 no termina cuando existen archivos. Termina cuando puedes explicar qué representa cada archivo, qué problema estás resolviendo y por qué aún no corresponde diseñar persistencia.